

Lab4: RISC-V 单周期处理器的设计与实现

1. 实验简介

在本实验室中，将使用原理图输入的方式，实现 32 位的 RISC-V 单周期处理器 singleriscv，指令集采用 32 位基本集 RV32I 的子集。

本实验要求修改和生成新的指令译码器和 ALU 两个 IP，并集成到预先给定的原理图中。然后将加载一个测试程序，并检查指令是否工作。接下来将以往试验中的 BCD 转二进制模块、二进制转 BCD 模块和显示模块等主要部件重新组合形成一个简单计算器 calcriscv。接下来，编写汇编程序实现加、减、乘等三种计算器功能，I/O 操作采用程序查询方式。

在本实验结束时，应该彻底了解 RISC-V 单周期处理器的数据通路和控制器，简单内存映射 I/O 外设的内部操作，了解汇编语言编程以及完整的硬件系统设计流程。

在开始本实验之前，应该非常熟悉课本和讲义中描述的 RISC-V 处理器的单周期实现。为了实验学习效果，已经给出了包括参考原理图的全部项目文件，本实验将在此基础上修改。本实验实现了除了 LB/LH/LBU/LHU/SB/SH/FENCE/CSR 这些指令之外的所有 RV32I 指令。

本实验有三个主要目标：

- 1) 理解与参考教科书中实现的不同处，增添的 xor / xori / sra / srai / sltu / blt / bge / bltu / bgeu / lui / jalr / auipc 等指令的控制器和数据通路的修改。
- 2) 删除参考原理图中的 Controller 模块，替换重新生成的 IP 并完成处理器连线。
- 3) 基于单周期处理器完成简单计算器设计，可实现十进制加、减、乘的功能，编写计算器汇编程序并测试通过。

本实验的地址映射和 I/O 寄存器地址如下，这些定义和 RISC-V 汇编模拟器 RARS 模拟器保持一致，只在 16 位空间中进行操作，32 位中的高 16 位会被忽略。

地址映射：

0x0000-0x0FFF：指令空间

0x1000-0x17FF：数据空间

0x7F00-0x7FFF：I/O 空间

其他：保留

I/O 寄存器地址：

PORTA: 0x7F00，只读，寄存器低 4 位保存 4 个按钮 btnL, btnC, btnR, btnU 通过同步器后的输入值。（按钮 btnD 定义为全局 Reset 信号）

PORTB: 0x7F10，只读，寄存器中保存 16 个开关 sw[15:0]经过 BCD2bin 转换后的 16 位二进制。低 14 位有效，高 2 位补零。

PORTC: 0x7F20，读写，寄存器中保存计算结果的低 14 位二进制经过 bin2BCD 转换通过数码管模块显示。

PORTD: 0x7FFC，读写，寄存器中保存值的低 16 位二进制输出 led[15:0]，驱动 16 个发光二极管。

SystemVerilog 模拟中虚拟停机寄存器的 I/O 地址

PORTD/HALT: 0x7FFC, 读写, 当数据地址是 0x7FFC, 同时写数据为 0x3 时, SystemVerilog 模拟停止。

2. 目标计算器的规范说明

2.1 功能描述: 用按钮选择适当的计算并将结果输出到 4 个 7 段数码管上 (复用 Lab1)。计算: 实现 2 位十进制的加、减、乘的计算功能。要求用原理图重新实现 RISC-V 单周期处理器, 并在此基础上实现减法功能。用 RISC-V 汇编程序实现乘法运算和所有结果的输出。

2.2 按钮输入的功能描述:

btnR: 乘法键, 将累加器 x5 的数值乘以输入 SW 经 BCD 转换后的 PORTB 数值, 结果写回累加器。。

btnC: 减法键, 将累加器 x5 的数值减去输入 SW 经 BCD 转换后的 PORTB 数值, 结果写回累加器。

btnL: 加法键, 将累加器 x5 的数值加上输入 SW 经 BCD 转换后的 PORTB 数值, 结果写回累加器。

btnU: 等号键, 将计算结果输出到 PORTC 并通过 BCD 转换后显式。

btnD: 复位键, 单周期处理器从 0x0000 开始取指执行。

2.3 其他输入输出功能描述:

采用 4 位十进制 BCD 输入和 4 位十进制输出。

采用 16 个开关, SW[15:0], 每四个开关分别为十进制 BCD 的千、百、十、个位输入, 通过 BCD2bin 转换电路转换成 16 位二进制, 进行加法、减法和乘法运算。

计算结果为 16 位二进制, 通过 Bin2BCD 的转换电路转换成 4 位十进制 BCD 输出。

3. RISC-V 汇编模拟器 RARS

RARS 模拟器是在 MARS MIPS 模拟器的基础上开发的支持 RISC-V 指令系统的汇编级模拟器, 采用 java 开发。下载网址:

<https://github.com/TheThirdOne/rars/releases>

3.1 下载并在 Windows 环境中直接运行 rars.jar, 需要安装 java8 JRE 或 JDK。

打开 test riscv.asm 汇编程序, 或者重编写新的汇编程序。

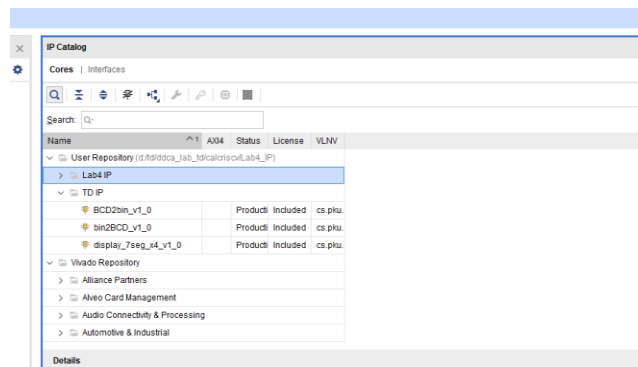
3.2 RARS 模拟器的 Run->Assemble 进行汇编过程, 然后可以运行或逐行调试, 可观察寄存器和内存变化。内存窗口下方可选择不同段的内存显示。

3.3 RARS 模拟器的 Settings->Memory Configuration... 中选择 Compact, Text at address 0, 此时程序段.text 段默认为 0x0000 开始, 数据段.data 默认为 0x1000 地址开始, 内存映射输入输出 MMIO 段默认为 0x7F00~0x7FFF。

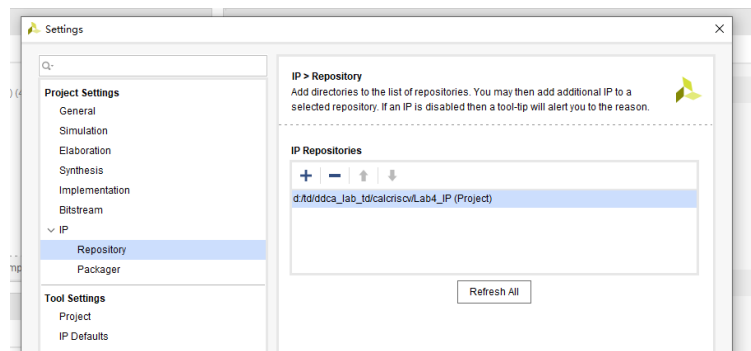
3.4 调试成功后，RARS 模拟器的 File->Dump Memory... 可以导出 SystemVerilog 语言 \$readmemh() 函数可以识别的 16 进制文本指令文件，Dump Format 选择 Hexadecimal text。生成 riscvtest.txt 文件还会用来生成 ROM 内容

4. calcriscv 单周期处理器和简单计算器

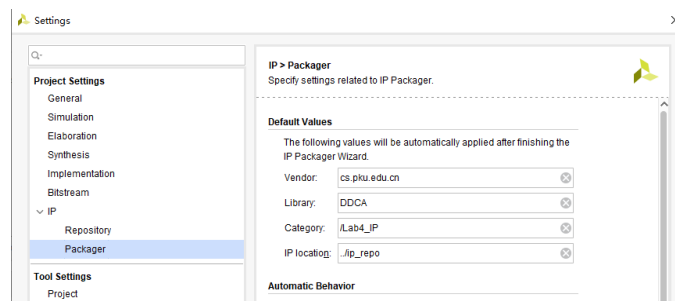
实验给出了单周期 RISC-V 的基本模块和原理图。将 calcriscv 目录复制到新的实验目录中。利用 Manage IP 工具打开 Lab4_IP 库，删除 bin2BCD、BCD2Bin、Display_7seg_x4 这三个 IP，并将之前自己实验生成的 IP 加入到 Lab4_IP 库中。如下图所示。



1) 用 vivado 打开 calcriscv_bd 目录下的项目文件 calcriscv_bd.xpr，将 IP 库设置成新的 Lab4_IP 库，应该共 14 个 IP 如下图所示。

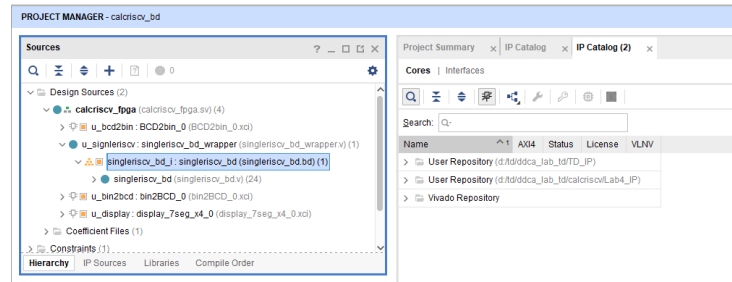


2) 在 src/ 目录中找到 controller.sv 文件，将模块名和所有输入输出信号添加 <姓名缩写> 的后缀，并相应修改内部逻辑。生成新的 SystemVerilog，controller_<姓名缩写>.sv。在 ip/ 目录下建立两个项目生成对应的 IP，Category 设置成 Lab4_IP。如下图所示。



用 Mange IP 工具将新生成的 IP 加入 Lab4_IP 库中，共 16 个 IP。

3) 在 Sources 窗口下，找到并双击鼠标打开 singleriscv_bd 原理图。



在 IP Category 中双击 **bin2BCD**、**BCD2Bin**、**Display_7seg_x4** 这三个 IP 并按 OK 重新生成 IP 实例模块（注意不要改_0 结尾名称，否则需要在 **calcriscv_fpga.sv** 中更改了相应模块名）。

4）在完全理解原理图的功能和连线意义之后，删除如图的模块 **controller**，替换成生成的 **Contrller_<名字缩写>**，然后重新连线。

注意与参考教科书相比，本设计在右下方多出了 3 个二选一模块，新增 2 个控制信号。

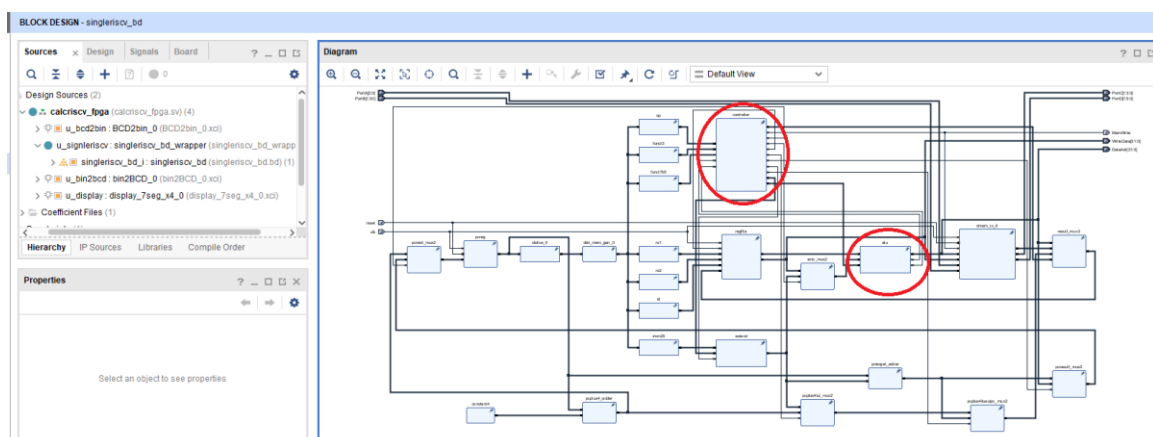
pcplus4lui_mux2 模块用来选择 **pc+4** 输出，还是立即数扩展 **extend** 模块的 **lui** 指令在 **utype** 信号控制下的高 20 位立即数输出。该模块由 **Jump** 信号控制。**Jump** 是 1 时表示 **jal** 指令 **pc+4** 回写 **rd**，否则输出 **lui** 的结果回写 **rd**。

pcplus4luiipc_mux2 模块用来选择 **AUIPC** 指令的结果，将 **extend** 模块在 **utype** 控制下的高位 20 位立即数与 **PC** 相加回写 **rd**。该模块由新增加的 **AUIPC** 控制信号控制，是 0 时选择 **pcpluslui_mux2** 的结果写回 **rd**（**jal** 或 **lui**），是 1 时 **AUIPC** 结果写回。

pcresult_mux2 模块用来支持 **jalr** 指令，将 **ALU (rs1+imm12)** 结果作为 **PCNext** 值。该模块由新增的 **JumpR** 信号控制。**JumpR** 是 1 时表示 **jalr** 指令，是 0 时则表示 **jal** 指令，将 **PCTarget = PC+Imm** 作为下一条指令地址。

原 **dmem** 模块更新为 **dmem_io** 模块，新增 **PortA**, **PortB**, **PortC**, **PortD** 等 4 个输入输出，并内部实现 4 个端口的读写功能。

原 **alu** 模块将 **ALUControl** 信号增加到[3:0]，高位表明是否是 **Sub** 操作，增加输出信号 **lessthan** 表明比较结构是否 **A<B**，用来在 **controller** 里面判断 **branch** 是否 **taken**。



更改完后。

鼠标点击 **singleriscv_bd**，在 **Tools** 菜单中选择 **Generate Output Products...**，生成项目文件。

Run Synthesis 进行综合。

Run Behavioral 进行模拟，如果正确会输出 **Simulation Successful...**，否则看波形查错误。

Run Implementation 并生成 bit 文件，下载编程 FPGA。如果正确，led 低 2 位会亮。拨动 sw 开关，会将 BCD 十进制数加 3 并显式。

5. 测试程序

本实验将使用下面的简单程序测试基本的功能：

#	RISC-V Assembly	Description	Address	Machine Code
main:	addi x2, x0, 5	# x2 = 5	0	00500113
	addi x3, x0, 12	# x3 = 12	4	00C00193
	addi x7, x3, -9	# x7 = (12 - 9) = 3	8	FF718393
	or x4, x7, x2	# x4 = (3 OR 5) = 7	C	0023E233
	and x5, x3, x4	# x5 = (12 AND 7) = 4	10	0041F2B3
	add x5, x5, x4	# x5 = (4 + 7) = 11	14	004282B3
	beq x5, x7, end	# shouldn't be taken	18	02728863
	slt x4, x3, x4	# x4 = (12 < 7) = 0	1C	0041A233
	beq x4, x0, around	# should be taken	20	00020463
	addi x5, x0, 0	# shouldn't happen	24	00000293
around:	slt x4, x7, x2	# x4 = (3 < 5) = 1	28	0023A233
	add x7, x4, x5	# x7 = (1 + 11) = 12	2C	005203B3
	sub x7, x7, x2	# x7 = (12 - 5) = 7	30	402383B3
	lui x3, 1	# x3 = 0x1000	34	000011B7
	sw x7, 12(x3)	# [0x100c] = 7	38	0071A623
	addi x3, x3, 8	# x3 = 0x1008	3C	00818193
	lw x2, 4(x3)	# x2 = [0x100c] = 7	40	0041A103
	add x9, x2, x5	# x9 = (7 + 11) = 18	44	005104B3
	jal x3, end	# jump to end, x3 = 0x4C	48	008001EF
	addi x2, x0, 1	# shouldn't happen	4C	00100113
end:	addi x2, x2, -4	# x2 = (7 - 4) = 3	50	FFC10113
	addi x3, x0, 8	# x3 = 8	54	00800193
	srli x3, x3, 3	# x3 = 8 >> 3 = 1	58	0031D193
	slli x3, x3, 15	# x3 = 1 << 15 = 0x8000	5C	00F19193
	auipc x8, 0	# x8 = 0x60	60	00000417
	jalr x1, x8, 8	# jump to 0x68 x1 = 0x68	64	008400E7
	addi x9, x3, -0x100	# x9 = 0x7f00	68	F0018493
	lw x6, 0x10(x9)	# x6 = [0x7f10]	6C	0104A303
	addi x6, x6, 3	# x6 = x6 + 3	70	00330313
	sw x6, 0x20(x9)	# [0x7f20] = x6	74	0264A023
	sw x2, -4(x3)	# [0x7ffc] = 3	78	FE21AE23
done:	bgeu x8, x2, done	# infinite loop	7C	00247063

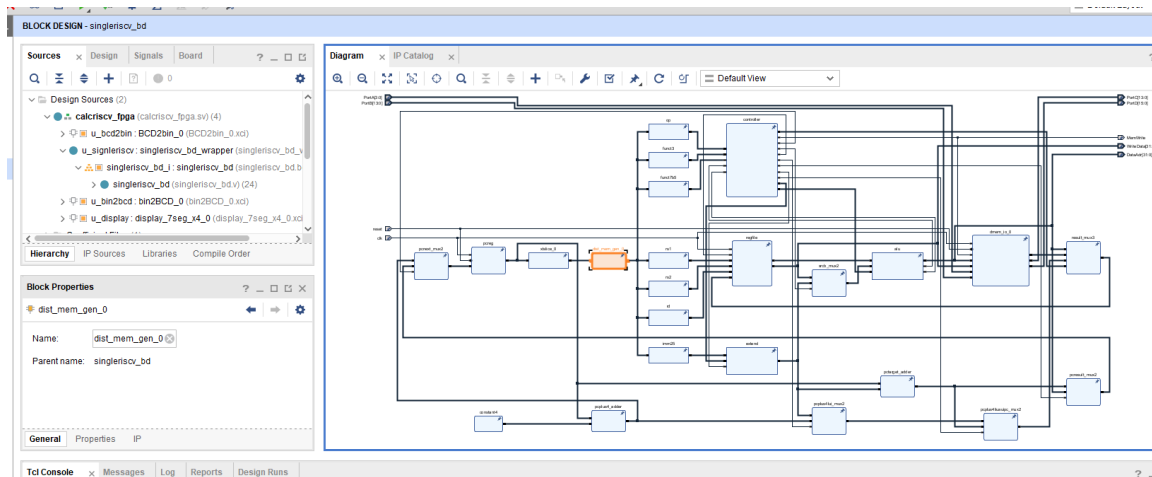
Figure 1. RISC-V 汇编程序 src/testriscv.s

这个代码可以在实验材料里面找到，可以用 RARS 汇编模拟器模拟。用 RARS 模拟器导出指令段的 16 进制文本文件 testriscv.txt，复制到 lab4 子目录中 src，用于 SystemVerilog 模拟。可见于 calcriscv_tb.sv 的 imem.sv 中的 \$readmemh() 函数。注意在模拟时 **需要将该文件按 Add Source 以 simulation 文件加入项目中**。

选择 calcriscv_tb.sv 并进行模拟，如果模拟正确，会在 Console 窗口中显示 simulate success 字样，在 0x7FFC 地址上写入了 3。

6. 生成指令 ROM 模块 imem_fpga

在原理图设计中集成了 Xilinx Distributed Memory Generator 工具生成 **特定的可综合的 ROM 模块, imem_fpga**，位置如下图所示。



需要采用**手动**的方式将 **testriscv.txt** 改写成如下的 **testriscv.coe** 文件。建议安装 **vscode** 或 **vim** 编辑软件。**加入前面 4 行后，在所有十六进制后加入 “,”号，最后一行加 “;”号。**

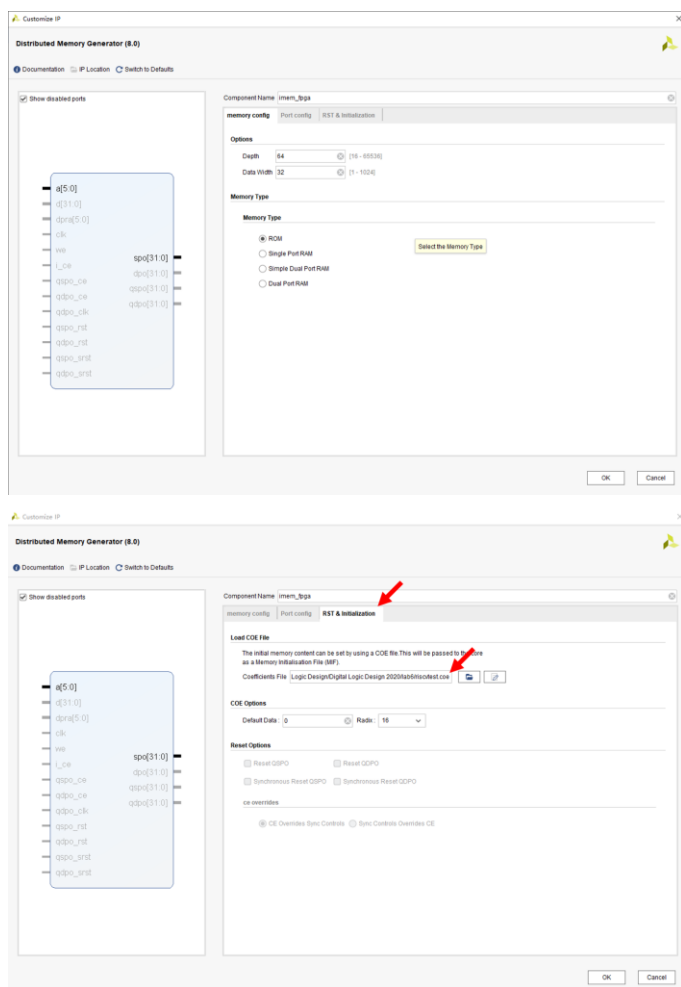
利用 **vim** 软件的块操作功能，将光标移到第一行指令的最后，**Ctrl+v** 进入块编辑模式，利用键盘上下左右键选中最后一列，按 **Shift+a**，输入“,”逗号，按 **Esc** 键退出，选中的所有列后面就加入了逗号。然后将最后一行的逗号改成“;”分号，保存退出即可。形成的文件如下：

```
; Sample initialization file for a
; 32-bit wide by 256 deep ROM
memory_initialization_radix = 16;
memory_initialization_vector =
00500113,
00c00193,
ff718393,
0023e233,
0041f2b3,
.....
00000417,
008400e7,
f0018493,
0104a303,
00330313,
0264a023,
fe21ae23;
00247063;
```

Figure 2. RISC-V 汇编程序 testriscv.asm 的 testriscv.coe 文件

每次修改程序，双击原理图中的 ROM 模块，如不改 COE 文件名，直接按 OK。

也可在第三个标签栏中更改 COE 文件名为自己生成的程序 16 进制文本程序，然后按 OK，生成新的 ROM IP。



这里有个坑，在后面自己写汇编的时候如果指令数量较多，一定要在memory config里把Depth调大，否则可能会有部分指令没有被存储到ROM里，导致FPGA运行异常

注意：每次更改 ROM 中的程序，都需要重新运行 Generate Output Products 步骤，并重新综合、模拟、实现、生成 bit 文件，生成新的 ROM 模块。

7. calcriscv 的综合和 FPGA 实验

综合最上层模块 calcriscv_fpga.sv。注意这个模块和用于模拟的 calcriscv_tb.sv 模块的区别，去掉了一些不必要的用于模拟的输出输出信号。

查看综合报告，确保没有错误发生，然后下载到 fpga 板上运行。当按下 btnD 后，发光二极管的最低 2 位 ld[1:0]亮，其他不亮，模拟正确。

将 sw 设置成 BCD 码，Reset 后数码管会显示加 3 的结果

8. calcriscv 单周期处理器的指令添加和计算器汇编的编写调试

并编写计算器的汇编程序，采用查询方法检测输入的变化，参考过程如下：

- 复位从 0x0000 开始执行指令
- 将作为累加器的寄存器 x5 和作为命令记录的寄存器 x6 写成 0
- 循环检测 porta 和 portb 的值是否改变，不改变循环检测。

- 如果 portb 的值改变了，将该值写入临时寄存器 x7 中，并写入输出寄存器 portc 中显示。
- 如果 porta 的值改变了，将该输入值写入临时寄存器 x8 中，并等待 porta 的值变为 0，否则循环等待。
- 如果 porta 的值变为 0，说明按钮已经抬起，判断 x6 寄存器是否为 0，为 0 将 x8 中的值写入 x6，将 x7 写入 x5，进入等待另一个操作数的阶段。
- 循环计算开始地址：循环判断 portb 是否改变，如果 portb 的值改变了，将该值写入临时寄存器 x7 中，并写入输出寄存器 portc 中显示
- 当检测再次 porta 的值再次变化，将该值写入 x8 中，循环等待变回 0 后，判断 x6 中的值进行转移到相应计算，将累加器 x5 的值和 x7 中的值进行操作，结果写入累加器 x5 中，并将结果写入 portc 显示。
- 回到循环计算开始地址。

参考教科书中的原图

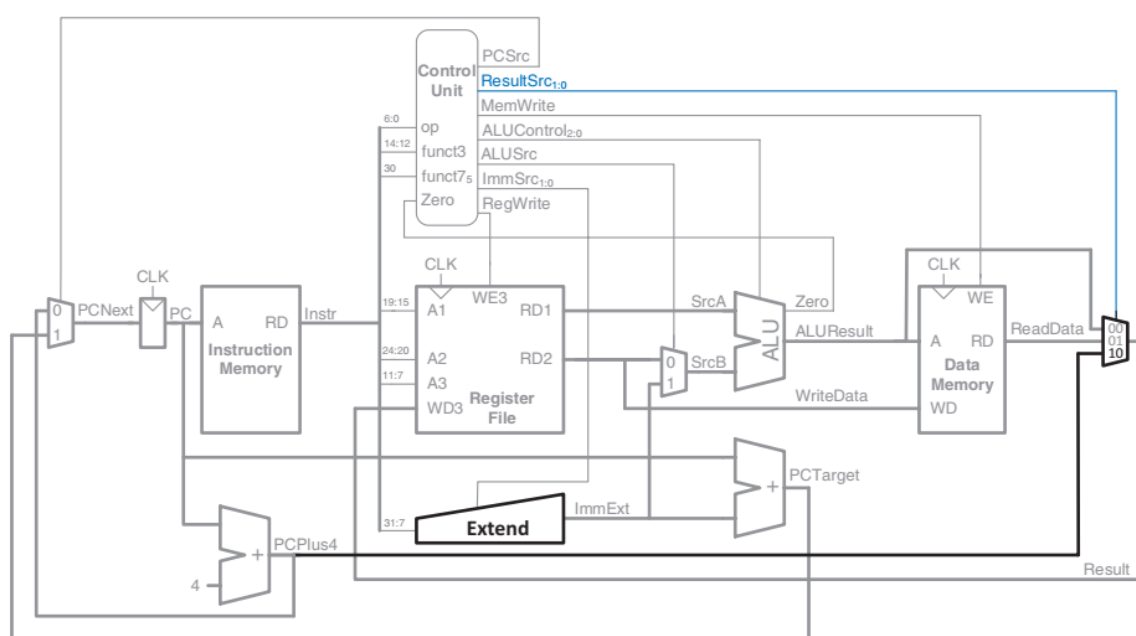


Figure 7.15 Enhanced datapath for jal

参考教科书中的主译码器

Table 7.6 Main Decoder truth table enhanced to support jal

Instruction	Opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
lw	0000011	1	00	1	0	01	0	00	0
sw	0100011	0	01	1	1	xx	0	00	0
R-type	0110011	1	xx	0	0	00	0	10	0
beq	1100011	0	10	0	0	xx	1	01	0
I-type ALU	0010011	1	00	1	0	00	0	10	0
jal	1101111	1	11	x	0	10	0	xx	1